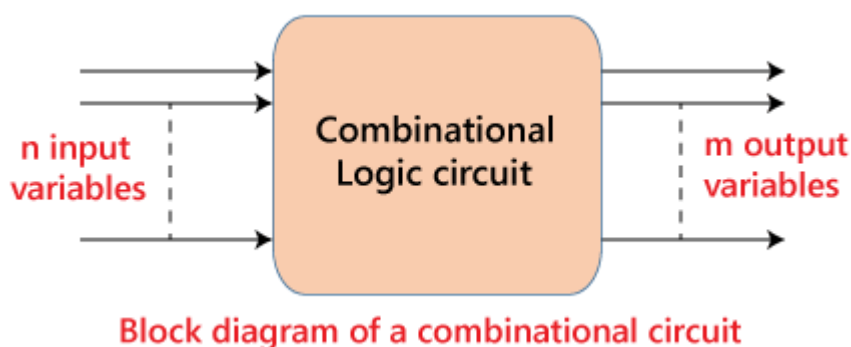


Combinational Logic circuits

The combinational logic circuits are the circuits that contain different types of logic gates. Simply, a circuit in which different types of logic gates are combined is known as a **combinational logic circuit**. The output of the combinational circuit is determined from the present combination of inputs, regardless of the previous input. The input variables, logic gates, and output variables are the basic components of the combinational logic circuit. There are different types of combinational logic circuits, such as Adder, Subtractor, Decoder, Encoder, Multiplexer, and De-multiplexer.

There are the following characteristics of the combinational logic circuit:

- At any instant of time, the output of the combinational circuits depends only on the present input terminals.
- The combinational circuit doesn't have any backup or previous memory. The present state of the circuit is not affected by the previous state of the input.
- The n number of inputs and m number of outputs are possible in combinational logic circuits.



The ' n ' input variable comes from the external source while the ' m ' output variable goes to the external destination. In many applications, the source or destinations are storage registers.

Half Adder

The half adder is a basic building block having two inputs and two outputs. The adder is used to perform OR operation of two single bit binary numbers. The **carry** and **sum** are two output states of the half adder.

Full Adder

The half adder is used to add only two numbers. To overcome this problem, the full adder was developed. The full adder is used to add three 1-bit binary numbers A, B, and carry C. The full adder has three input states and two output states i.e., sum and carry.

Half Subtractors

The half subtractor is also a building block of subtracting two binary numbers. It has two inputs and two outputs. This circuit is used to subtract two single bit binary numbers A and B. The '**diff**' and '**borrow**' are the two output state of the half adder.

Full Subtractors

The Half Subtractor is used to subtract only two numbers. To overcome this problem, full subtractor was designed. The full subtractor is used to subtract three 1-bit numbers A, B, and C, which are **minuend**, **subtrahend**, and **borrow**, respectively. The full subtractor has three input states and two output states i.e., diff and borrow.

Multiplexers

The multiplexer is a combinational circuit that has n-data inputs and a single output. It is also known as the **data selector** which selects one input from the inputs and routes it to the output. With the help of the selected inputs, one input line from the n-input lines is selected. The enable input is denoted by E, which is used in cascade.

De-multiplexers

A De-multiplexer performs the reverse operation of a multiplexer. The de-multiplexer has only one input, which is distributed over several outputs. One output line is selected at a time by selecting lines. The input is transmitted to the selected output line.

Decoder

A decoder is a combinational circuit having n inputs and to a maximum of $m = 2^n$ outputs. The decoder is the same as the de-multiplexer. The only difference between de-multiplexer and decoder is that in the decoder, there is no data input. The decoder performs an operation that is completely opposite of an encoder.

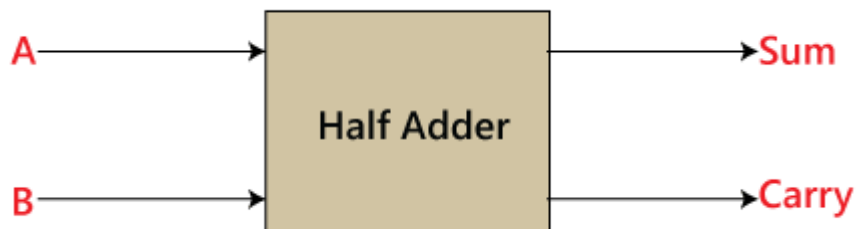
Encoder

The encoder is used to perform the reverse operation of the decoder. An encoder having n number of inputs and m number of outputs is used to produce m-bit binary code which is related to the digital input number. The encoder takes the digital word and converts it into another digital word

Half Adder

The Half-Adder is a basic building block of adding two numbers as two inputs and produce out two outputs. The adder is used to perform OR operation of two single bit binary numbers. The **augend** and **addend** bits are two input states, and '**carry**' and '**sum**' are two output states of the half adder.

Block diagram



Truth Table

Inputs		Outputs	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

In the above table,

1. 'A' and 'B' are the input states, and 'sum' and 'carry' are the output states.
2. The carry output is 0 in case where both the inputs are not 1.
3. The least significant bit of the sum is defined by the 'sum' bit.

The SOP form of the sum and carry are as follows:

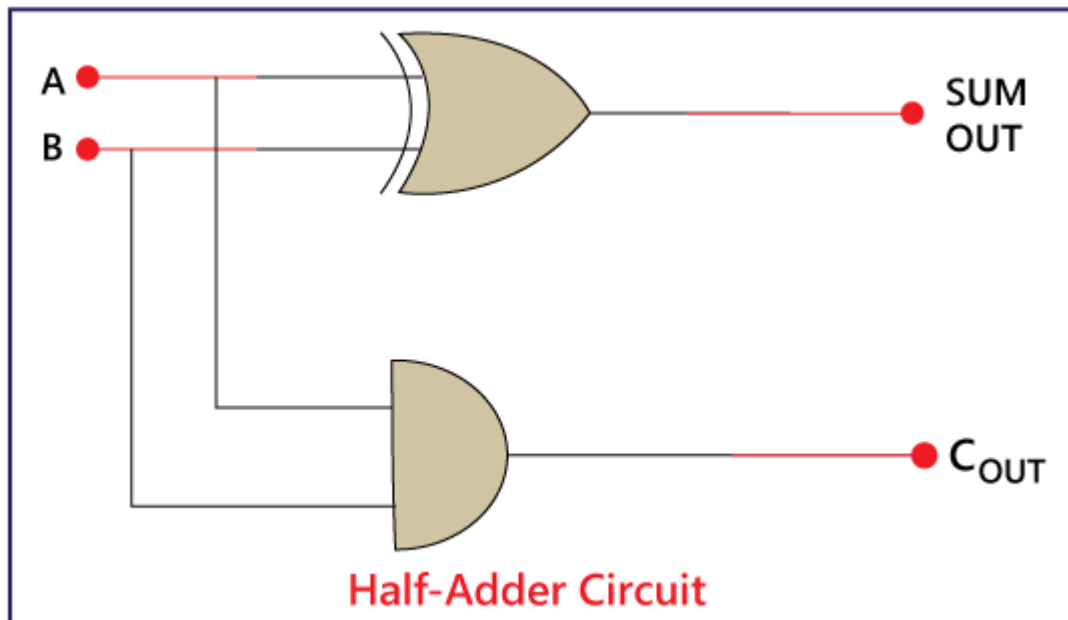
$$\text{Sum} = x'y + xy'$$

$$\text{Carry} = xy$$

Construction of Half Adder Circuit:

Half-Adder logical circuit:

So, the Half Adder is designed by combining the 'XOR' and 'AND' gates and provide the sum and carry.



There is the following **Boolean expression** of **Half Adder circuit**:

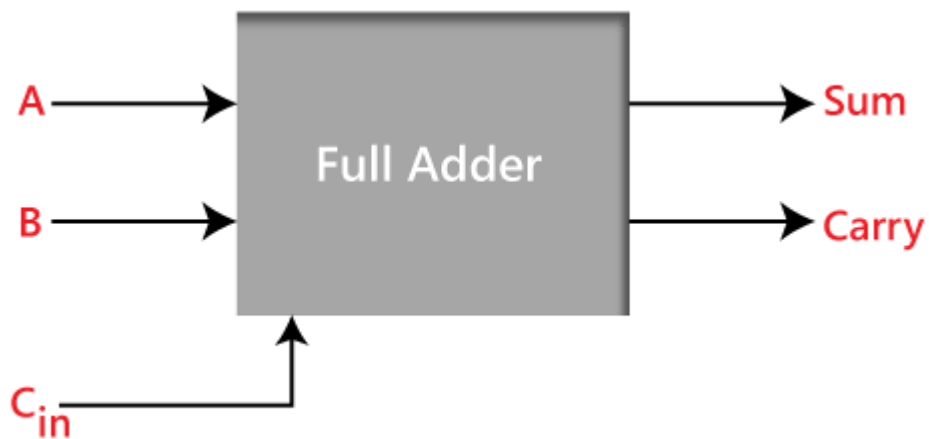
$$\text{Sum} = A \text{ XOR } B \quad (A+B)$$

$$\text{Carry} = A \text{ AND } B \quad (A.B)$$

Full Adder

The half adder is used to add only two numbers. To overcome this problem, the full adder was developed. The full adder is used to add three 1-bit binary numbers A, B, and carry C. The full adder has three input states and two output states i.e., sum and carry.

Block diagram

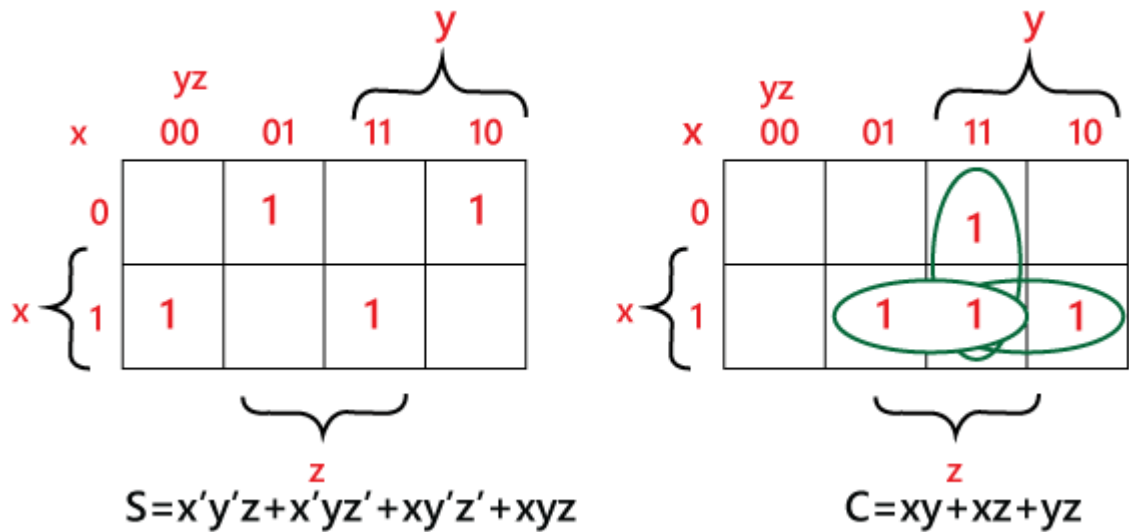


Truth Table

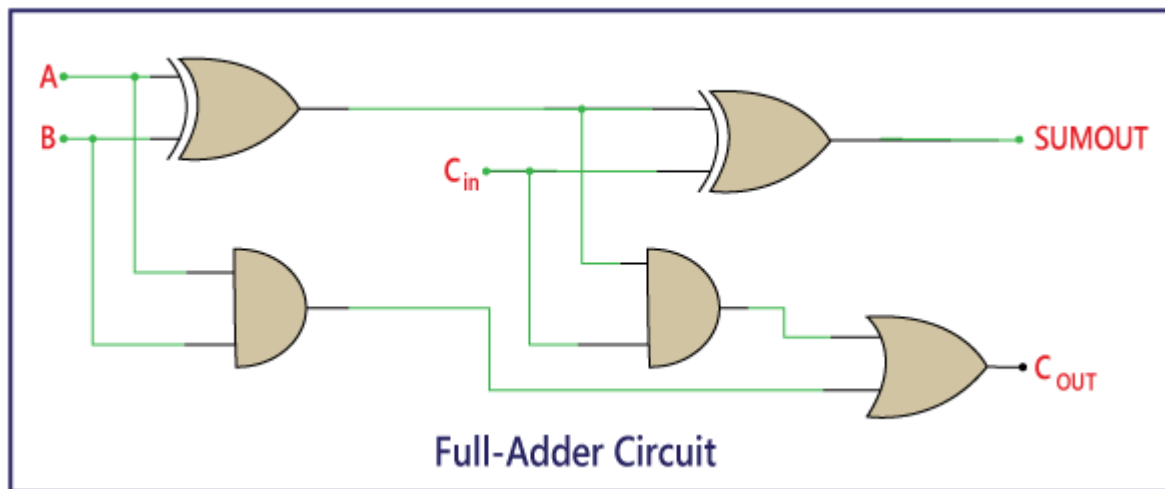
Inputs			Outputs	
A	B	C _{in}	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

In the above table,

1. 'A' and 'B' are the input variables. These variables represent the two significant bits which are going to be added
2. 'C_{in}' is the third input which represents the carry. From the previous lower significant position, the carry bit is fetched.
3. The 'Sum' and 'Carry' are the output variables that define the output values.
4. The eight rows under the input variable designate all possible combinations of 0 and 1 that can occur in these variables.
5. The SOP form can be obtained with the help of K-map as:



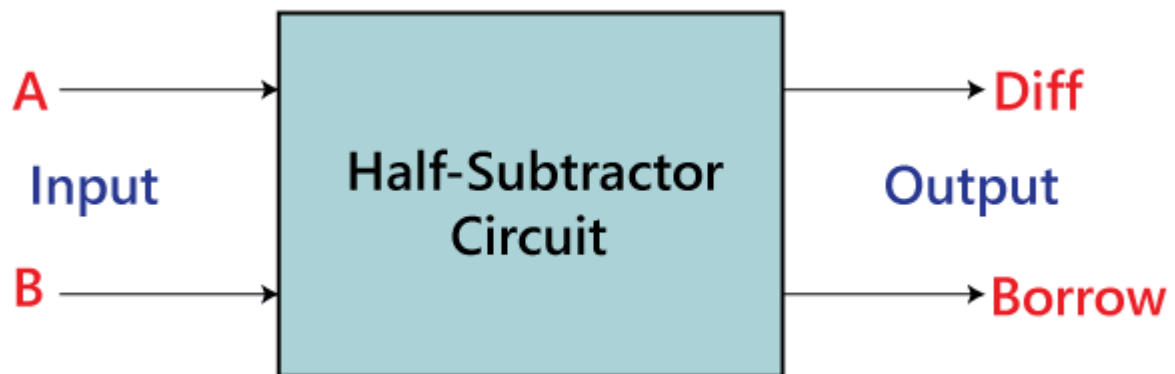
- 6.
7. Sum = $x' y' z + x' y z' + x y' z' + x y z$
Carry = $xy + xz + yz$
8. The full adder logic circuit can be constructed using the '**AND**' and the '**XOR**' gate with an **OR** gate.



Half Subtractor

The half subtractor is also a building block for subtracting two binary numbers. It has two inputs and two outputs. This circuit is used to subtract two single bit binary numbers A and B. The '**diff**' and '**borrow**' are two output states of the half subtractor.

Block diagram



Truth Table

Inputs		Outputs	
A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

The SOP form of the **Diff** and **Borrow** is as follows:

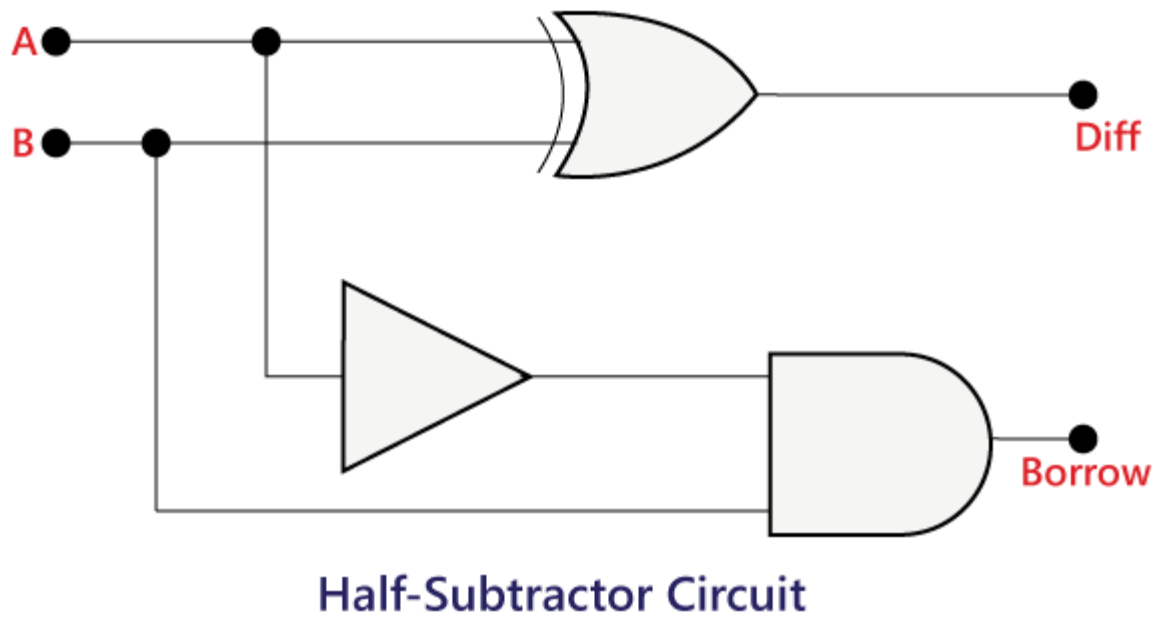
Diff=

$A'B + AB'$

Borrow = $A'B$

Half-Subtractor logical circuit

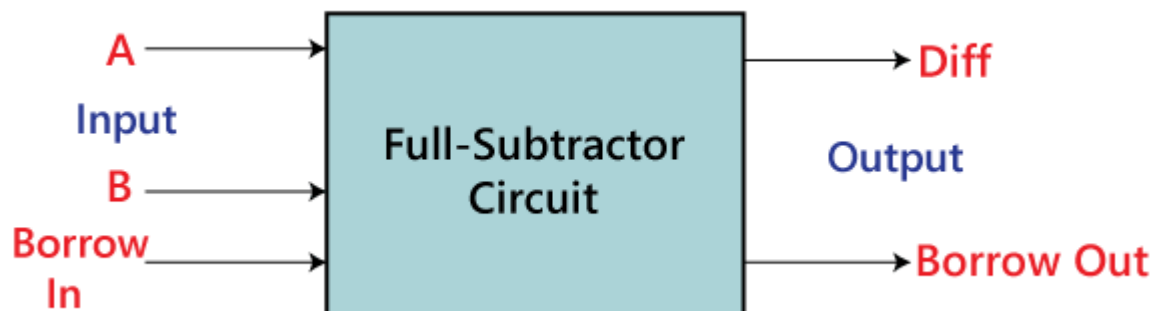
So, the Half Subtractor is designed by combining the 'XOR', 'AND', and 'NOT' gates and provide the Diff and Borrow.



Full Subtractor

The Half Subtractor is used to subtract only two numbers. To overcome this problem, a full subtractor was designed. The full subtractor is used to subtract three 1-bit numbers A, B, and C, which are minuend, subtrahend, and borrow, respectively. The full subtractor has three input states and two output states i.e., diff and borrow.

Block diagram

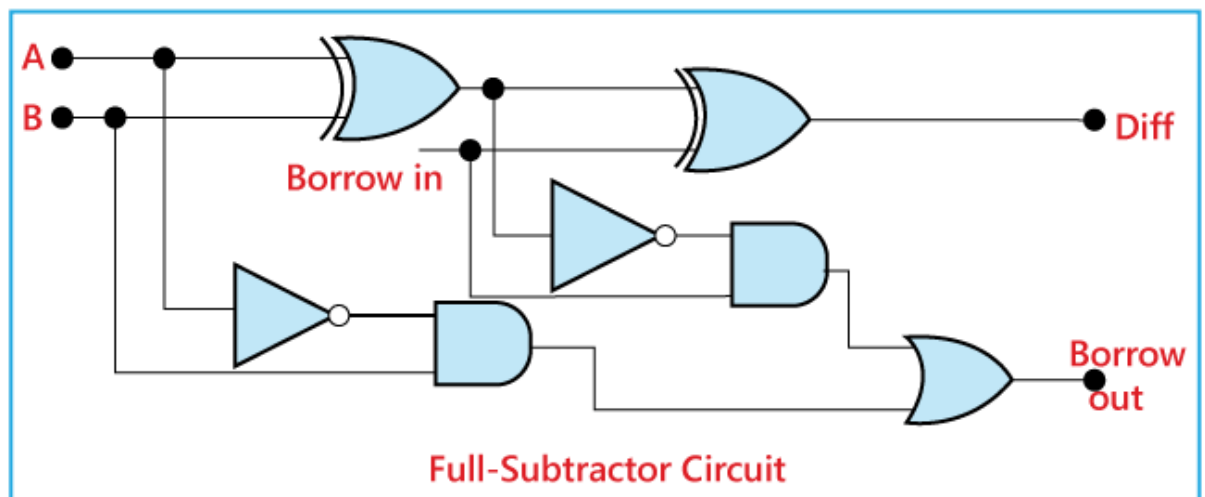


Truth Table

Inputs			Outputs	
A	B	Borrow _{in}	Diff	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

In the above table,

- 'A' and 'B' are the input variables. These variables represent the two significant bits that are going to be subtracted.
- 'Borrow_{in}' is the third input which represents borrow.
- The 'Diff' and 'Borrow' are the output variables that define the output values.
- The eight rows under the input variable designate all possible combinations of 0 and 1 that can occur in these variables.
- The full subtractor logic circuit can be constructed using the '**AND**', '**XOR**', and **NOT gate** with an **OR gate**.



Binary Adder

The registers play an important role in performing the micro-operations. The registers hold the digital component and the data which performs the arithmetic operation. The

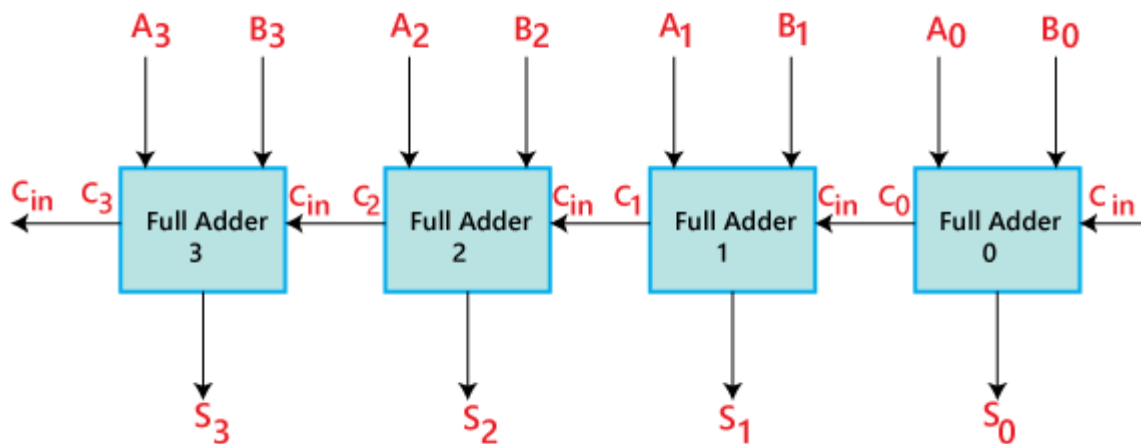
Binary Adder is a logical circuit which is used to perform the addition operation of two binary number of any length.

The Binary Adder is formed with the help of the Full-Adder circuit. The Full-Adders are connected in series, and the output carry of the first Adder will be treated as the input carry of the next Full-Adder.

N-Bit Parallel Adder

The Full Adder is used to sum two single-bit binary numbers with carry input. In digital calculation, we need to add two n-bit binary numbers rather than only single-bit binary numbers. For this purpose, we need to use n-bit parallel Adder. In order to get N-bit parallel adder, we cascade the n number of Full Adders. The carry output of the first Adder is treated as the carry input of the second Adder.

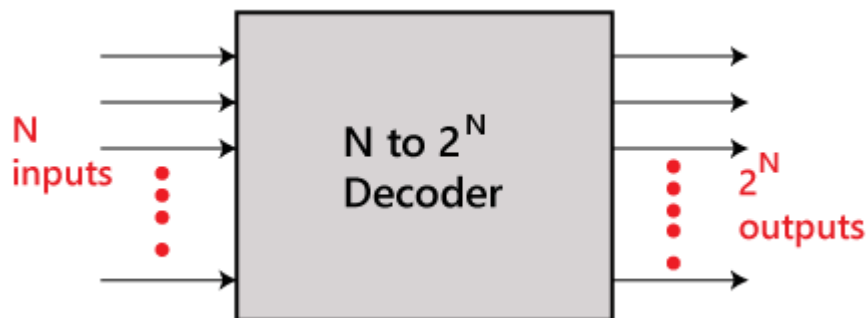
4-bit Binary Adder



- The 'A' and 'B' are the augend, and addend bits are defined by the subscript numbers. The subscripts start from right to left, and the lower-order bit is defined by subscript '0'.
- The C₀, C₁, C₂, and C₃ are the carry inputs which are connected together as a chain using Full Adder. The C₄ is the carry output produced by the last Full-Adder.
- The C_{out} of the first Adder is connected as the C_{in} of the next Full-Adder.
- The S₀, S₁, S₂, and S₃ are the sum outputs that produce the sum of augend and addend bits.
- The inputs for the input variable 'A' and 'B' are fetched from different source registers. For example, the bit for the input variable 'A' comes from register 'R1', and a bit for the input variable 'B' comes from register 'R2'.
- The outcome produced by adding both input variables is stored into either third register or to one of the source registers.

Decoder

The combinational circuit that change the binary information into 2^N output lines is known as **Decoders**. The binary information is passed in the form of N input lines. The output lines define the 2^N -bit code for the binary information. In simple words, the **Decoder** performs the reverse operation of the **Encoder**. At a time, only one input line is activated for simplicity. The produced 2^N -bit output code is equivalent to the binary information.

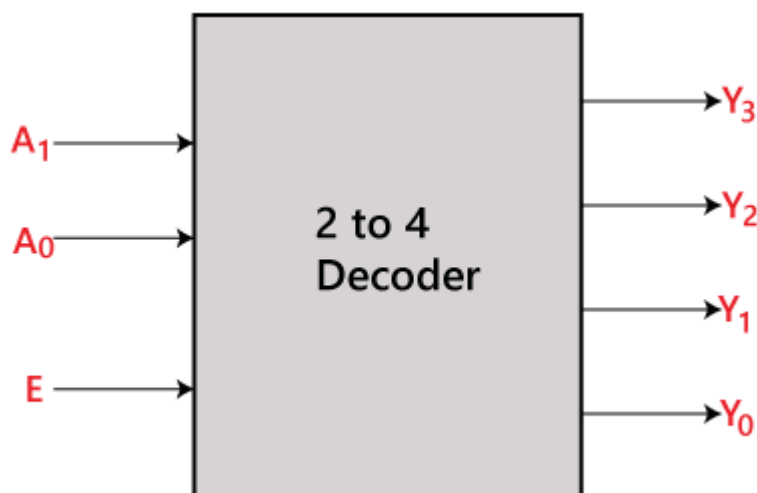


There are various types of decoders which are as follows:

2 to 4 line decoder:

In the 2 to 4 line decoder, there is a total of three inputs, i.e., A_0 , and A_1 and E and four outputs, i.e., Y_0 , Y_1 , Y_2 , and Y_3 . For each combination of inputs, when the enable 'E' is set to 1, one of these four outputs will be 1. The block diagram and the truth table of the 2 to 4 line decoder are given below.

Block Diagram:



Truth Table:

Enable	INPUTS		OUTPUTS			
E	A ₁	A ₀	Y ₃	Y ₂	Y ₁	Y ₀
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

The logical expression of the term Y₀, Y₁, Y₂, and Y₃ is as follows:

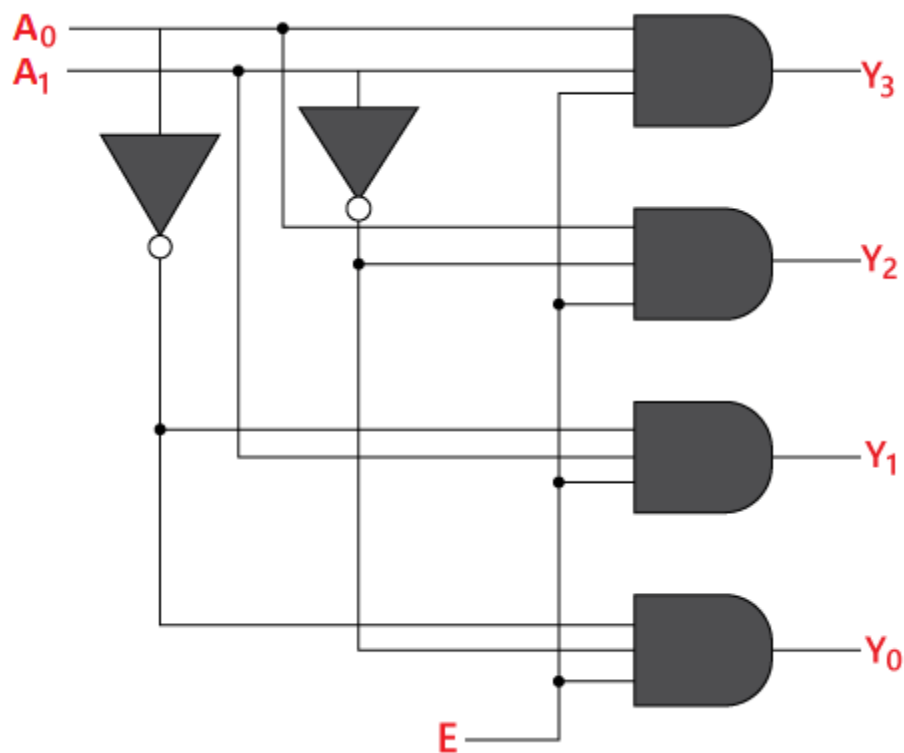
$$Y_3 = E \cdot A_1 \cdot A_0$$

$$Y_2 = E \cdot A_1 \cdot A_0'$$

$$Y_1 = E \cdot A_1' \cdot A_0$$

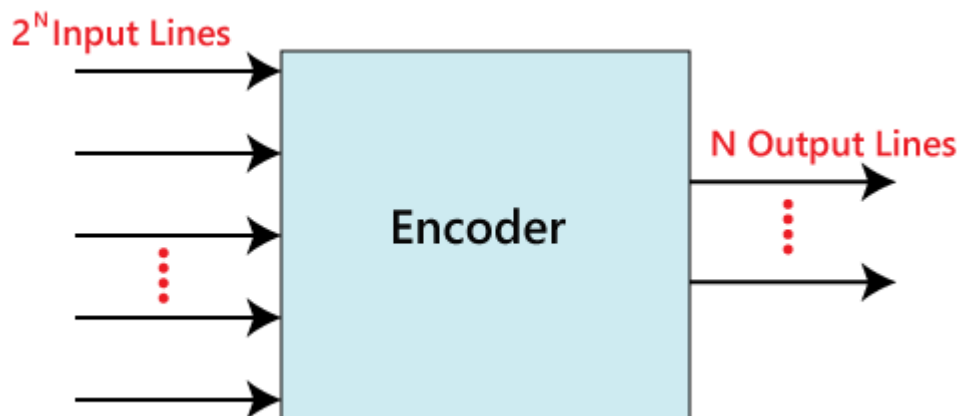
$$Y_0 = E \cdot A_1' \cdot A_0'$$

Logical circuit of the above expressions is given below:



Encoders

The combinational circuits that change the binary information into N output lines are known as **Encoders**. The binary information is passed in the form of 2^N input lines. The output lines define the N-bit code for the binary information. In simple words, the **Encoder** performs the reverse operation of the **Decoder**. At a time, only one input line is activated for simplicity. The produced N-bit output code is equivalent to the binary information.

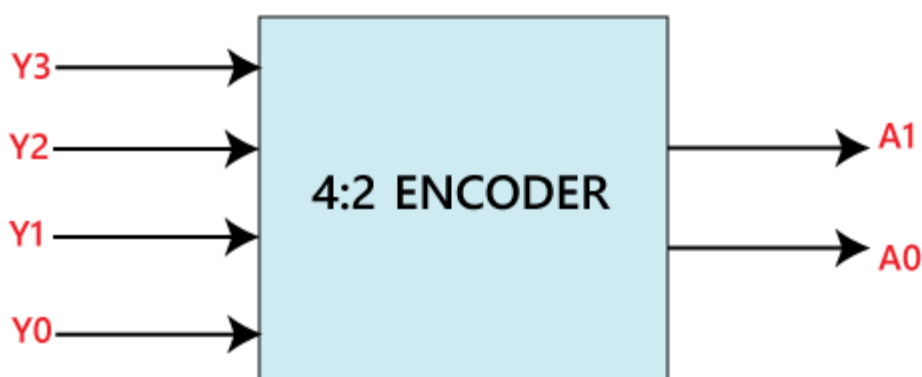


There are various types of encoders which are as follows:

4 to 2 line Encoder:

In 4 to 2 line encoder, there are total of four inputs, i.e., Y_0 , Y_1 , Y_2 , and Y_3 , and two outputs, i.e., A_0 and A_1 . In 4-input lines, one input-line is set to true at a time to get the respective binary code in the output side. Below are the block diagram and the truth table of the 4 to 2 line encoder.

Block Diagram:



Truth Table:

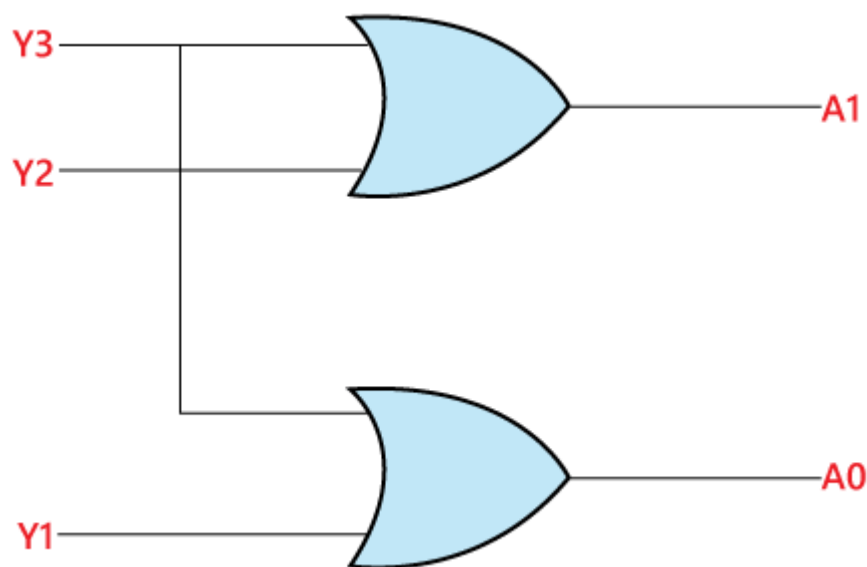
INPUTS				OUTPUTS	
Y_3	Y_2	Y_1	Y_0	A_1	A_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

The logical expression of the term A_0 and A_1 is as follows:

$$A_1 = Y_3 + Y_2$$

$$A_0 = Y_3 + Y_1$$

Logical circuit of the above expressions is given below:



Multiplexer

A multiplexer is a combinational circuit that has 2^n input lines and a single output line. Simply, the multiplexer is a multi-input and single-output combinational circuit. The binary information is received from the input lines and directed to the output line. On the basis of the values of the selection lines, one of these data inputs will be connected to the output.

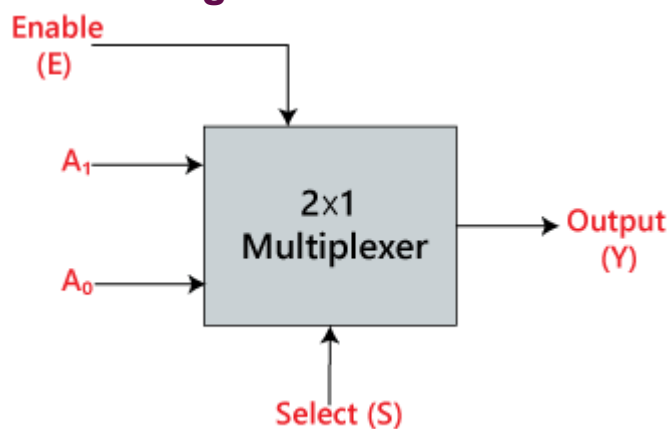
Unlike encoder and decoder, there are n selection lines and 2^n input lines. So, there is a total of 2^N possible combinations of inputs. A multiplexer is also treated as **Mux**.

There are various types of the multiplexer which are as follows:

2×1 Multiplexer:

In 2×1 multiplexer, there are only two inputs, i.e., A_0 and A_1 , 1 selection line, i.e., S_0 and single outputs, i.e., Y . On the basis of the combination of inputs which are present at the selection line S^0 , one of these 2 inputs will be connected to the output. The block diagram and the truth table of the 2×1 multiplexer are given below.

Block Diagram:



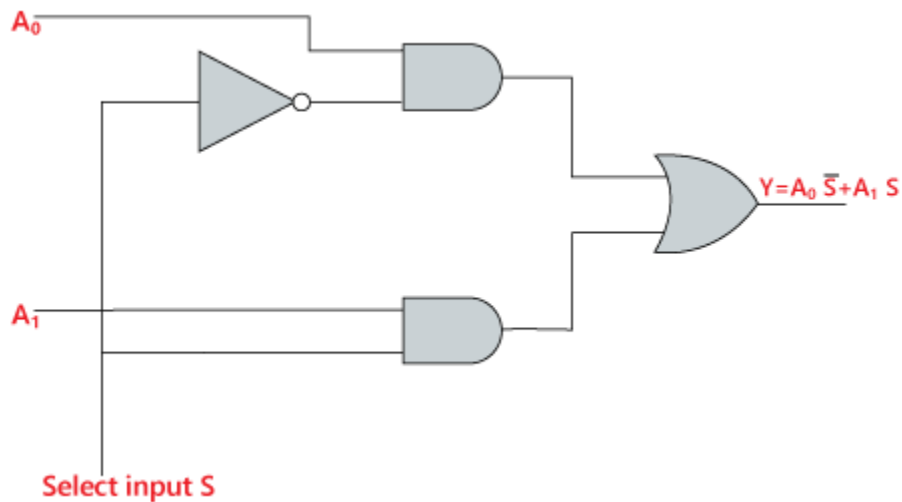
Truth Table:

INPUTS	Output
S_0	Y
0	A_0
1	A_1

The logical expression of the term Y is as follows:

$$Y = S_0' \cdot A_0 + S_0 \cdot A_1$$

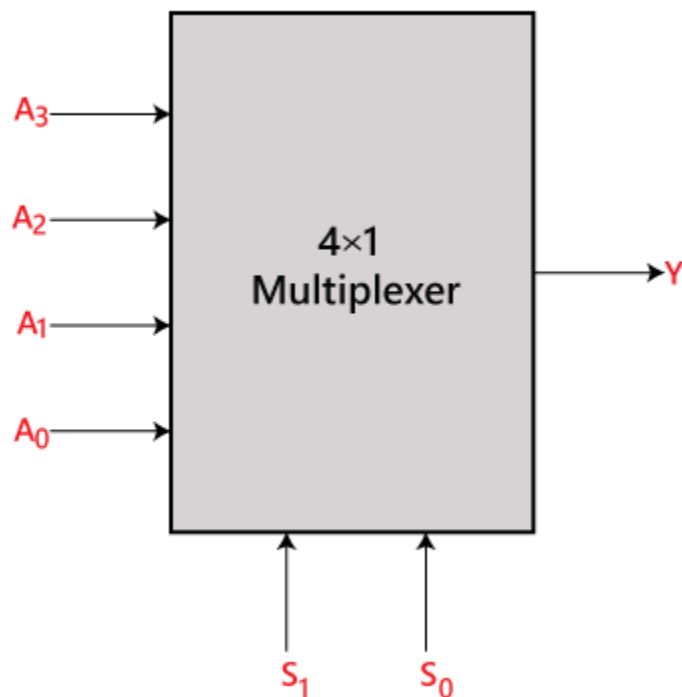
Logical circuit of the above expression is given below:



4×1 Multiplexer:

In the 4×1 multiplexer, there is a total of four inputs, i.e., A_0 , A_1 , A_2 , and A_3 , 2 selection lines, i.e., S_0 and S_1 and single output, i.e., Y . On the basis of the combination of inputs that are present at the selection lines S_0 and S_1 , one of these 4 inputs are connected to the output. The block diagram and the truth table of the 4×1 multiplexer are given below.

Block Diagram:



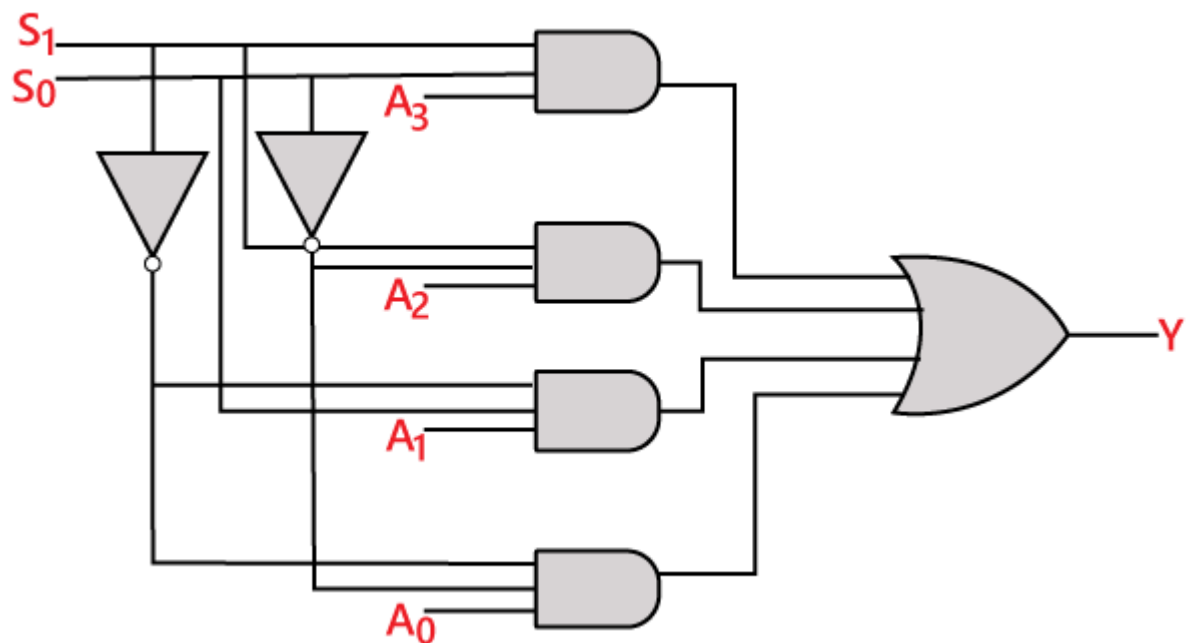
Truth Table:

INPUTS		Output
S_1	S_0	Y
0	0	A_0
0	1	A_1
1	0	A_2
1	1	A_3

The logical expression of the term Y is as follows:

$$Y = S_1' S_0' A_0 + S_1' S_0 A_1 + S_1 S_0' A_2 + S_1 S_0 A_3$$

Logical circuit of the above expression is given below:



De-multiplexer

A De-multiplexer is a combinational circuit that has only 1 input line and 2^N output lines. Simply, the multiplexer is a single-input and multi-output combinational circuit. The information is received from the single input lines and directed to the output line. On the basis of the values of the selection lines, the input will be connected to one of these outputs. De-multiplexer is opposite to the multiplexer.

Unlike encoder and decoder, there are n selection lines and 2^n outputs. So, there is a total of 2^n possible combinations of inputs. De-multiplexer is also treated as **De-mux**.

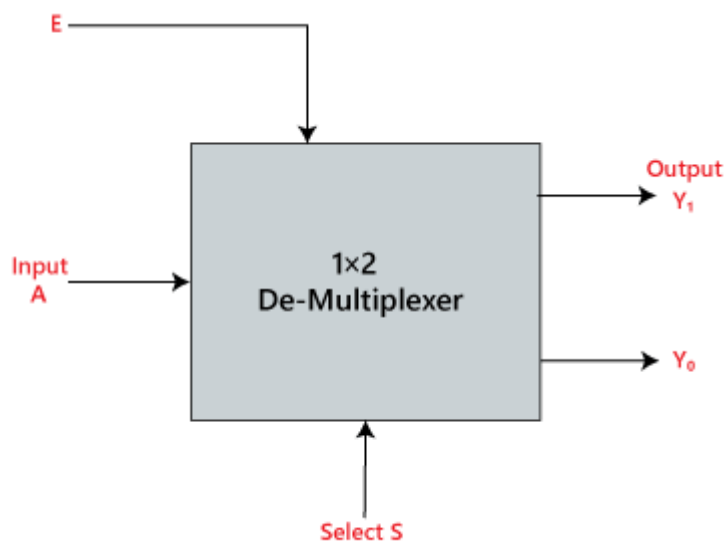
There are various types of De-multiplexer which are as follows:

1×2 De-multiplexer:

In the 1 to 2 De-multiplexer, there are only two outputs, i.e., Y_0 , and Y_1 , 1 selection lines, i.e., S_0 , and single input, i.e., A . On the basis of the selection value, the input will be connected to one of the outputs. The block diagram and the truth table of the 1×2 multiplexer are given below.

Backward Skip 10sPlay VideoForward Skip 10s

Block Diagram:



Truth Table:

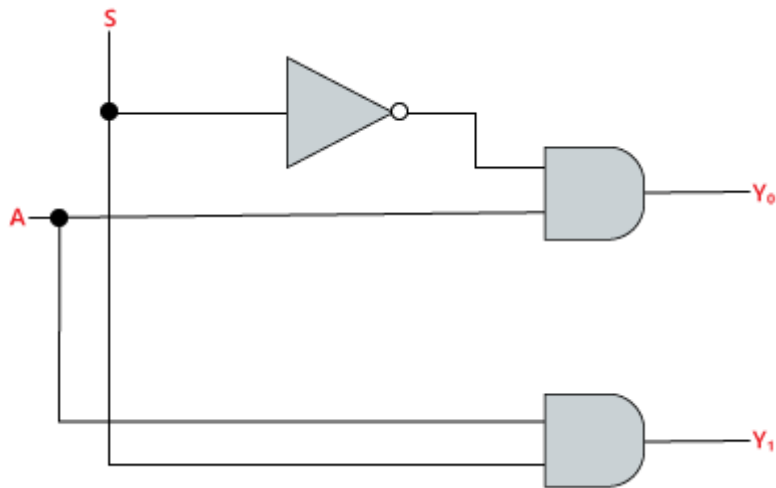
INPUTS	Output	
S_0	Y_1	Y_0
0	0	A
1	A	0

The logical expression of the term Y is as follows:

$$Y_0 = S_0' \cdot A$$

$$Y_1 = S_0 \cdot A$$

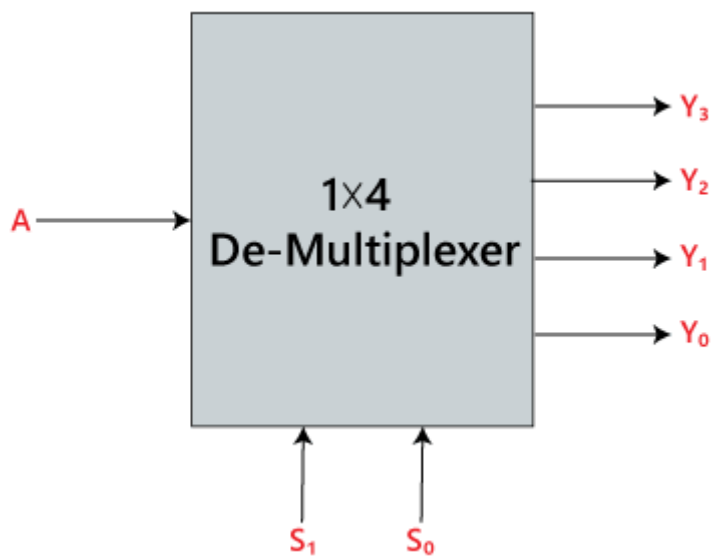
Logical circuit of the above expressions is given below:



1×4 De-multiplexer:

In 1 to 4 De-multiplexer, there are total of four outputs, i.e., Y_0 , Y_1 , Y_2 , and Y_3 , 2 selection lines, i.e., S_0 and S_1 and single input, i.e., A. On the basis of the combination of inputs which are present at the selection lines S_0 and S_1 , the input be connected to one of the outputs. The block diagram and the truth table of the 1×4 multiplexer are given below.

Block Diagram:



Truth Table:

INPUTS		Output			
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	A
0	1	0	0	A	0
1	0	0	A	0	0
1	1	A	0	0	0

The logical expression of the term Y is as follows:

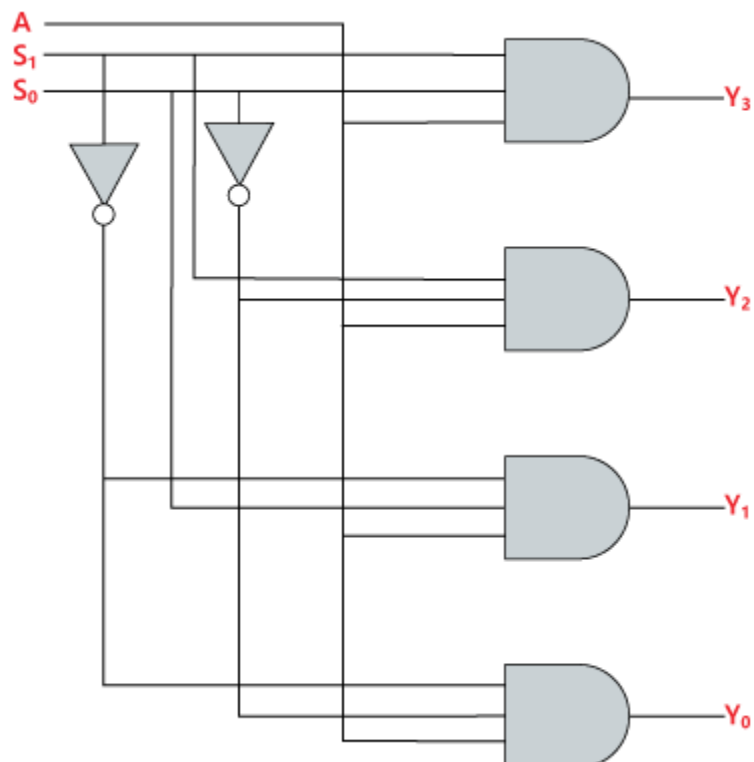
$$Y_0 = S_1' S_0' A$$

$$Y_1 = S_1' S_0 A$$

$$Y_2 = S_1 S_0' A$$

$$Y_3 = S_1 S_0 A$$

Logical circuit of the above expressions is given below:



BCD to Seven Segment Decoder

The Binary Coded Decimal (BCD) encoding method uses binary patterns to encode each decimal value (from 0 to 9), generally of 4-bits.

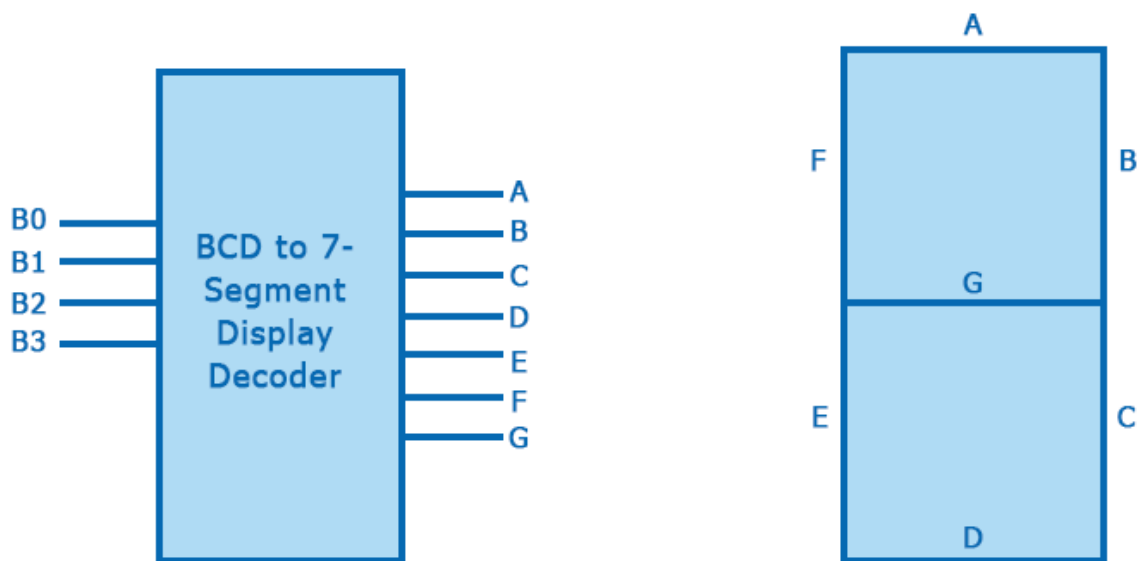
In contrast, a seven-segment display is an electrical device that displays Hexadecimal digits using seven Light Emitting Diodes (LEDs) arranged in a specific pattern (common cathode or common anode type) (in this case, decimal numbers, as input, are BCD, i.e., 0-9).

7-Segment LED displays are of two types:

1. **Common Cathode Type (CCT)** - This form of the display works by connecting all seven of the LEDs' cathodes to ground or -Vcc (thus, common cathode), which causes the LEDs to display numbers when a "HIGH" signal is applied to each anode.
2. **Common Anode Type (CAT)** - The seven LEDs in this display have all seven of their anodes linked to a battery or +Vcc, and when a 'LOW' signal is applied to each individual cathode, the LEDs display numbers.

The separate LED segments of a seven-segment display are not directly powered. However, for it to function, *we first convert our decimal number to its BCD equivalent signal, and then a BCD to seven-segment decoder converts that signal into the form supplied to the seven-segment display.*

The seven-segment decoder, which has four input lines and seven output lines (a, b, c, d, e, f, and g), receives this BCD (A, B, C, and D) input. The output is provided to a seven-segment LED display that shows the decimal number depending on the inputs.



The truth table of Common Cathode Type BCD to the seven-segment decoder is given below:

Decimal Number	INPUT Lines				OUTPUT Lines						
	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

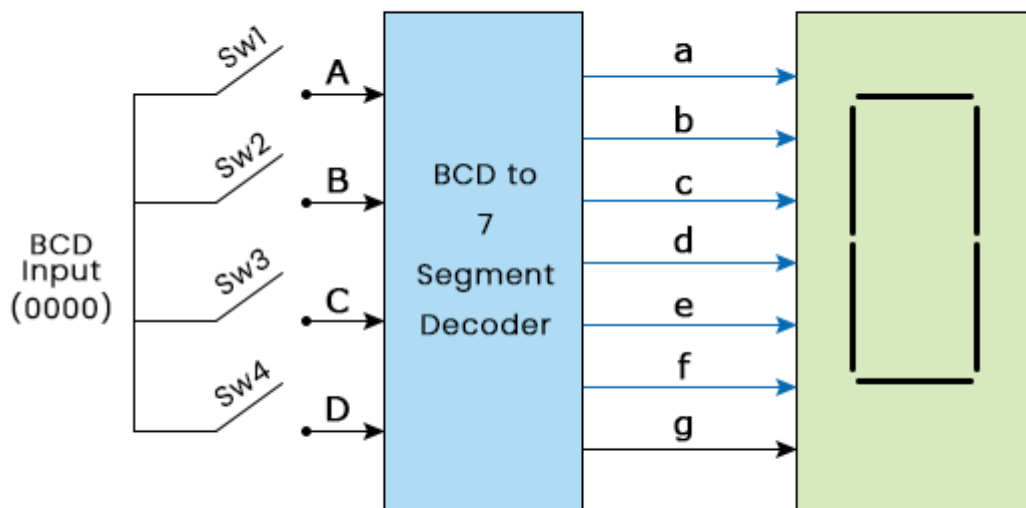
Important points:

1. For a seven-segment LED display of the Common Anode type, all we need to do is swap all of the '0's and '1's on the output side, i.e., for a, b, c, d, e, f, and g, swap all of the '1's for '0's, and use k-map to solve.
2. In the above truth table, the combinations of inputs represent the digits from 0 to 1.
3. The range of BCD numbers is 0 to 9, and the rest of the inputs are invalid.

Let us now understand the few inputs:

For Decimal Number = 0

The input lines for Zero are $A = 0$, $B = 0$, $C = 0$, and $D = 0$ and in the output lines, a to f are 1 and only g = 0. So, the g LED display will not glow, and the rest of the LEDs will glow, and we get zero on display.



For Decimal Number = 2

The input lines for two are $A = 0$, $B = 0$, $C = 1$, and $D = 0$; in the output lines, a, b, d, e, and g are 1 and c and f are zero. So, only a, b, d, e, and g will glow, and we get two on display.

